RIPHAH INTERNATIONALUNIVERSITY GULBERG GREEN CAMPUS, ISLAMABAD.

Assignment

Course: Software Re-Engineering

Submitted by: Sabahat Qadeer (47235)

Submitted to: Ma'am Sadia Rashid

Date of submission: 05-11-2025

Department of BSSE

Riphah International University, Islamabad.

Questions

1. Identify **code smells** from each scenario.
2. Which **design principles** are violated?
3. Suggest **specific refactoring** to fix them.

Scenario 1

s

```
public class Document {
    public virtual void Print() { Console.WriteLine("Printing..."); }
    public virtual void SendFax() { Console.WriteLine("Fax sent"); }
}

public class PDFDocument : Document {
    public override void SendFax() {
        // Not supported for PDF
        throw new NotSupportedException("Fax not applicable");
    }
}
```

1. Identify code smells

Refused Bequest / Inappropriate Inheritance

The subclass PDFDocument inherits a method SendFax() that it cannot logically support, forcing it to throw an exception. This indicates an incorrect inheritance relationship.

2. Violated Design Principles

- **Liskov Substitution Principle (LSP)**: Subclasses should be substitutable for their parent classes without altering program correctness. PDFDocument violates this by throwing an exception instead of performing the inherited behavior.
- **Interface Segregation Principle (ISP)**: Classes should not be forced to depend on methods they do not use.

3. Refactoring

Refactoring Type: Extract Interface / Replace Inheritance with Composition

Instead of inheriting from a single class that contains both printing and faxing functionalities, the system should separate responsibilities using interfaces.

```
public interface IPrintable {
    void Print();
}

public interface IFaxable {
    void SendFax();
}

public class Document : IPrintable, IFaxable {
    public void Print() { Console.WriteLine("Printing..."); }
    public void SendFax() { Console.WriteLine("Fax sent"); }
}

public class PDFDocument : IPrintable {
    public void Print() { Console.WriteLine("Printing PDF..."); }
```

Result:

- PDFDocument now implements only the behavior it needs.
- No unnecessary inheritance or exceptions are required.
- Design now follows both LSP and ISP correctly.

Scenario 2

```
public class RegularCustomerService {
    public double calculateDiscount(double total) {
        double discount = 0;
        if (total > 500) discount = total * 0.05;
        return discount;
    }
}

public class PremiumCustomerService {
    public double calculateDiscount(double total) {
        double discount = 0;
        if (total > 500) discount = total * 0.05;
        if (total > 1000) discount = total * 0.10;
        return discount;
    }
}
```

1. Identify Code Smells

- **Duplicate Code:** Both RegularCustomerService and PremiumCustomerService contain similar discount calculation logic (if (total > 500) discount = total * 0.05;). This repetition increases maintenance effort and makes the system harder to extend.
- **Lack of Abstraction:** There is no common abstraction for customer types. Each new type of customer requires a separate class with similar logic.

2. Design Principles Violated

- **Open/Closed Principle (OCP):** The design violates OCP because adding a new customer type (GoldCustomer, SilverCustomer) requires modifying or duplicating existing code instead of extending behavior through abstraction.
- **DRY Principle:** The discount logic for totals over 500 is repeated in both classes, which is a clear violation of the DRY principle.

3. Refactoring

Refactoring Technique*:* Introduce Abstraction (Interface or Abstract Class)

Step 1: Create a common abstraction for all customer types.

```
public interface CustomerService {
    double calculateDiscount(double total);
}

}
```

Step 2: Make each customer type implement this interface.

```
public class RegularCustomerService implements CustomerService {
    @Override
    public double calculateDiscount(double total) {
        return total > 500 ? total * 0.05 : 0;
    }
}
public class PremiumCustomerService implements CustomerService {
    @Override
    public double calculateDiscount(double total) {
        if (total > 1000) return total * 0.10;
        else if (total > 500) return total * 0.05;
        return 0;
    }}
```

Step 3: Use polymorphism to handle different customer types through the CustomerService interface, without modifying existing classes.

Result

- Code becomes extensible, new customer types can be added easily.
- No duplication of logic.
- The design now follows Open/Closed and DRY principles.

Scenario 3

```java
public double calculateShippingCost(String location, String method,
    boolean express, boolean fragile) {
    double cost = 0;
    if (location.equals("Domestic")) {
        if (method.equals("Air")) cost = 50;
        else if (method.equals("Road")) cost = 20;
        if (express) cost += 10;
        if (fragile) cost += 15;
    } else if (location.equals("International")) {
        if (method.equals("Air")) cost = 100;
        else if (method.equals("Sea")) cost = 70;
        if (express) cost += 30;
        if (fragile) cost += 25;
    }
    return cost;
}
```

1. Identify Code Smells

- **Long Method:** The method performs several calculations and conditions, making it lengthy and difficult to read.
- **Nested Conditionals:** Multiple if-else statements make the code complex and hard to maintain.
- **Magic Numbers:** Hard-coded numeric values like 50, 70, and 100 reduce readability and flexibility.
- **Lack of Extensibility:** To add a new location or shipping method, existing code must be changed.

2. Design Principles Violated

- **Single Responsibility Principle (SRP):**
  The method performs several different types of calculations instead of focusing on a single responsibility.
- **Open/Closed Principle (OCP):**
  The method is not open for extension , new shipping methods require editing existing code.

3. Refactoring

Refactoring Techniques Used:

- Extract Class / Strategy Pattern
- Replace Conditional with Polymorphism

```

interface ShippingStrategy {
    double calculateCost(boolean express, boolean fragile);
}

class DomesticAirShipping implements ShippingStrategy {
    public double calculateCost(boolean express, boolean fragile) {
        double cost = 50;
        if (express) cost += 10;
        if (fragile) cost += 15;
        return cost;
    }
}

class InternationalSeaShipping implements ShippingStrategy {
    public double calculateCost(boolean express, boolean fragile) {
        double cost = 70;
        if (express) cost += 30;
        if (fragile) cost += 25;
        return cost;
    }
}

```

Scenario 4

```java
public class Payment {
    private String cardNumber;
    private String expiryDate;
    private String cvv;

    public boolean validate() {
        if (cardNumber.length() != 16 || !cardNumber.matches("\\d+")) return false;
        if (!expiryDate.matches("\\d{2}/\\d{2}")) return false;
        if (cvv.length() != 3) return false;
        return true;
    }
}
```

1. Code Smells Identified

- **Primitive Obsession:**
  The class uses simple strings instead of proper data types for card information.
- **Hardcoded Logic:**
  Validation rules and patterns are directly written in the method.
- **Lack of Separation of Concerns:**
  The same class stores data and performs validation.

2. Design Principles Violated

- **Single Responsibility Principle (SRP):**
  The class handles both data storage and validation , two separate responsibilities.
- **Open/Closed Principle (OCP):**
  Any change in validation rules requires modifying the same class.

3. Refactoring to Fix the Problem

Refactoring Techniques Used:

- Extract Class
- Introduce Interface / Strategy Pattern

Step 1: Create an interface for validation

```
public interface PaymentValidator {
    boolean validate();
}
```

Step 2: Create a concrete validator for card payments.

```
public class CardPaymentValidator implements PaymentValidator {
    private String cardNumber;
    private String expiryDate;
    private String cvv;

    public CardPaymentValidator(String cardNumber, String expiryDate, String cvv)
{
        this.cardNumber = cardNumber;
        this.expiryDate = expiryDate;
        this.cvv = cvv;
    }

    @Override
    public boolean validate() {
        if (cardNumber.length() != 16 || !cardNumber.matches("\\d+")) return
false;
        if (!expiryDate.matches("\\d{2}/\\d{2}")) return false;
        if (cvv.length() != 3) return false;
        return true;
    }
}
```

Step 3: Payment class uses the validator

```
public class Payment {
    private PaymentValidator validator;

    public Payment(PaymentValidator validator) {
        this.validator = validator;
    }

    public boolean processPayment() {
        return validator.validate();
    }
}
```

Result After Refactoring

- Each class now has a single, clear purpose (SRP followed).
- The design is extensible**,** new payment methods (PayPal, Bank Transfer) can be added by creating new validators (OCP followed).
- Code becomes clean, modular, and easy to test.

Scenario 5

```java
public class Booking {
    private String customerName;
    private String checkInDate;
    private String checkOutDate;
    private String paymentMethod;
    private double totalAmount;

    public void calculateTotal(double roomRate, int nights) {
        totalAmount = roomRate * nights;
    }

    public void processPayment() {
        if (paymentMethod.equals("CreditCard")) {
            System.out.println("Processing via credit card...");
        } else if (paymentMethod.equals("PayPal")) {
            System.out.println("Processing via PayPal...");
        }
    }

    public void sendConfirmationEmail() {
        System.out.println("Email sent to " + customerName);
    }
}
```

1. Code Smells Identified

- **God Class:**
  The class performs multiple unrelated functions, calculations, payment processing, and email sending.
- **Conditional Logic:**
  Multiple if-else statements determine payment methods, reducing scalability.
- **Tight Coupling:**
  The class directly depends on specific payment types such as “CreditCard” and “PayPal”.

2. Design Principles Violated

- **Single Responsibility Principle (SRP):**
  The class violates SRP by managing multiple responsibilities.
- **Open/Closed Principle (OCP):**
  Adding a new payment method requires modifying the class.
- **Dependency Inversion Principle (DIP):**
  The class depends on concrete payment implementations instead of abstractions.

3. Refactoring

- Use Strategy Pattern to handle different payment types through an interface.
- Move Email Functionality to a separate EmailService class.
- Reduce Coupling by depending on abstract interfaces rather than specific classes.

```java

interface PaymentMethod {
    void processPayment();
}

class CreditCardPayment implements PaymentMethod {
    public void processPayment() {
        System.out.println("Processing via credit card...");
    }
}

class PayPalPayment implements PaymentMethod {
    public void processPayment() {
        System.out.println("Processing via PayPal...");
    }
```

```
}
class Booking {
    private String customerName;
    private double totalAmount;
    private PaymentMethod paymentMethod;

    public void calculateTotal(double roomRate, int nights) {
        totalAmount = roomRate * nights;
    }

    public void processPayment() {
        paymentMethod.processPayment();
    }}
```

Scenario 6

```
class ReportGenerator:
    def __init__(self):
        self.start_date = None
        self.end_date = None
        self.temp_graph = None

    def generate(self, report_type):
        if report_type == "Monthly":
            self.start_date = "01-10-2025"
            self.end_date = "31-10-2025"
        elif report_type == "Summary":
            self.temp_graph = self.create_graph()
        print("Generating " + report_type + " report")

    def create_graph(self):
        print("Creating temporary summary graph")
        return "GraphObject"
```

1. Identify Code Smells

God Class / Large Class: Class handling too many responsibilities (dates, graph creation, report generation).

Tight Coupling: The ReportGenerator directly depends on graph creation logic.

2. Design Principles Violated

- **Single Responsibility Principle (SRP):**

  The class performs several tasks instead of focusing on one responsibility.
- **Open/Closed Principle (OCP):**

  Adding a new report type requires modifying the existing class.
- **Dependency Inversion Principle (DIP):**

  The class directly depends on the creategraph() implementation.

3. Refactoring

- Extract Class
- Use Strategy Pattern
- Parameterize Dates

```
class GraphGenerator:
    def create_graph(self):
        print("Creating temporary summary graph")
        return "GraphObject"

class MonthlyReport:
    def __init__(self, start, end):
        self.start = start
        self.end = end

    def generate(self):
        print(f"Generating Monthly Report from {self.start} to {self.end}")

class SummaryReport:
    def __init__(self, graph):
        self.graph = graph

    def generate(self):
        print("Generating Summary Report with", self.graph.create_graph())

// Example usage
MonthlyReport("01-10-2025", "31-10-2025").generate()
SummaryReport(GraphGenerator()).generate()
```

Scenario 7

```
public class Product {
    public string Name { get; set; }
    public double Price { get; set; }
    public string Category { get; set; }
}

public class Cart {
    public void AddProduct(Product product) {
        Console.WriteLine("Added " + product.Name + " costing $" + product.Price);
    }

    public void ApplyDiscount(Product product, double discount) {
        Console.WriteLine("New price: $" + (product.Price - discount));
    }
}
```

1. Identify Code Smells

- **Feature Envy**: The Cart class is directly accessing Product details and performing price calculations that should belong to Product.
- **Low Cohesion**: Cart handles responsibilities that are not directly related to managing a collection of products (discount calculation).
- **Code Duplication Risk**: Discount logic might need to be reused elsewhere, but it's tied to Cart.
- **Primitive Obsession**: Using simple data types for money handling instead of a dedicated class or method.

2. Design Principles are Violated

- **SRP (Single Responsibility Principle):**
  The Cart class is responsible for both managing products and calculating discounts, two different tasks.
- **Encapsulation Principle:**
  The Cart class directly manipulates Product properties instead of using Product's own methods.
- **OCP (Open/Closed Principle):**
  Any change in discount logic requires modification of the Cart class.

3. Refactoring

- **Move Method:** Move ApplyDiscount logic into the Product class where price behavior belongs.
- **Encapsulate Field:** Allow Product to manage its own price and discount through methods.
- **Increase Cohesion:** Keep Cart focused on product management only (adding/removing products).

```
•
•  public class Product {
•      public string Name { get; set; }
•      public double Price { get; private set; }
•      public string Category { get; set; }
•
•      public Product(string name, double price, string category) {
•          Name = name;
•          Price = price;
•          Category = category;
•      }
•
•      public void ApplyDiscount(double discount) {
•          Price -= discount;
•          Console.WriteLine("New price of " + Name + ": $" + Price);
•      }
•  }
•
•  public class Cart {
•      public void AddProduct(Product product) {
•          Console.WriteLine("Added " + product.Name + " costing $" +
   product.Price);
•      }
•  }
```